

UNIVERSITY OF SCHOLARS

Bazar-Trace: Offline-First PWA Stock & Inventory Management System

Built with HTML, CSS, Vanilla JS, Node.js (Custom HTTP Server) & Oracle Database (Local)

PRESENTER

Arif Hossen

SUPERVISOR

Reduanul Bari Subhon sir

MENTOR

Israt Jahan miss

ID: 231010687 · CSE



What is Bazar-Trace?

- **Vanilla Single Page Application** — An SPA built purely with raw ES Modules, HTML5, and CSS variables (no React, Next.js, or Tailwind CSS).
- **Offline-First Sync Engine** — Continues recording transactions and looking up products even when the internet drops completely.
- **Role-Based Security** — Strict separation of admin management and staff registry options.
- **Mobile-First Design** — Optimized for low-end devices and simple mobile browsers.



Motivation & Problem Statement

- 1 **Internet Instability** — Retail shops in Bangladesh frequently suffer from network drops, rendering standard cloud tools unusable.
- 2 **High Overhead Costs** — Small store owners cannot afford expensive desktop setups or heavy software licenses.
- 3 **UI/UX Clutter** — Existing ERP and inventory portals are too complex for non-technical cashiers.
- 4 **Academic Constraint** — The challenge of building sophisticated offline sync and real-time canvas charting without heavy third-party libraries.



Project Goals

- 1 **Zero Downtime** — Enable cashiers to record Sales (OUT) and Stock received (IN) with zero latency during offline states.
- 2 **Conflict-Free Auto Synchronization** — Replay offline mutations in strict FIFO order on network recovery, handling sync retries idempotently.
- 3 **Hardware Integrations** — Access native camera modules for barcode scanning and trigger synthesized audio scan beeps.
- 4 **Resource Constraints Compliance** — Build the entire system under strict university guidelines (Vanilla CSS, raw JS, Oracle DB).



High-Level Architecture

SYSTEM ARCHITECTURE

CLIENT PWA

Vanilla HTML5 / CSS



RegExp Router + Lazy Modules



Offline-First API Client



IndexedDB Cache

HTTP



BACKEND — CUSTOM NODE.JS SERVER

http.createServer()



Custom Router + Middleware



Oracle DB (Local Native)

 Service Worker intercepts & caches the app shell (Stale-While-Revalidate)



Technology Stack

FRONTEND CORE

HTML5, CSS Variables layout system, ES6 JavaScript Modules

HARDWARE APIS

getUserMedia, BarcodeDetector API, Web Audio Synthesizer

DATABASE

Oracle XE 21c — local native · oracledb v6 Thin mode
(localhost:1521/XEPDB1)

PWA ENGINE

Service Worker (Stale-While-Revalidate), IndexedDB API

BACKEND — ZERO FRAMEWORK

Node.js (ESM, --watch) · Custom http server · Hand-built
router, middleware, CORS, body parser

INFRASTRUCTURE

Docker Compose (production only) · Git/GitHub



Functional Requirements

STAFF / COMMON

- Secure Login / Session validation
- Log transactions (Sale OUT / Stock Received IN)
- Search products (dynamic autocomplete)
- Scan barcodes with camera to autofill forms

ADMINISTRATOR

- Full Product Management (Create, Edit, Delete)
- Settings Directory (Register staff, Toggle status)
- Dashboard analytics (Stock counts, Canvas charts, Expiry alerts)



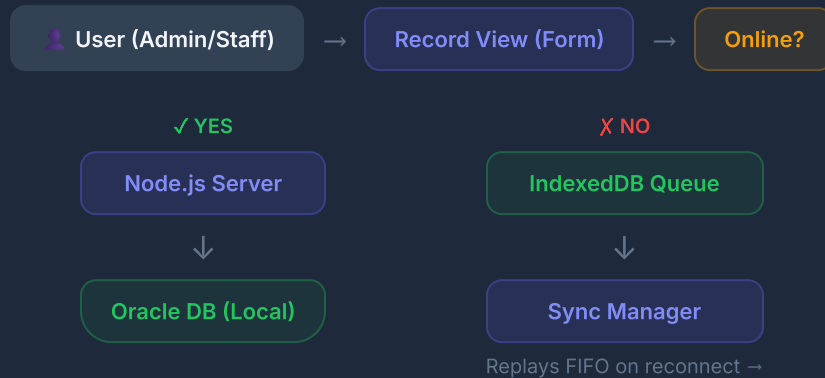
Non-Functional Requirements

- **Performance** — UI response time under 50ms; static bundle load under 200ms.
- **Availability** — 100% operational offline. App shell cached, allowing disconnections at any stage.
- **Reliability** — FIFO sync queue prevents duplicate inserts via unique transaction UUIDs (idempotency).
- **Security** — Cryptographic Bcrypt hashing for credentials; token-based JWT route authentication.
- **Usability** — Minimalist layouts, clear alert badges, automatic input formatting.



Data Flow Diagram — Level 1

TRANSACTION LIFECYCLE





Entity Relationship Diagram

USERS	
id	PK
email	UNIQUE
password_hash	
full_name	
role	
is_active	
created_at	

PRODUCTS	
id	PK
sku	UNIQUE
barcode	
name, category, unit	
cost_price, sell_price	
low_stock_threshold	
expiry_date	
created_by	FK → users

TRANSACTIONS	
id	PK
client_txn_id	UNIQUE
product_id	FK → products
user_id	FK → users
txn_type (IN/OUT)	
quantity, unit_price	
occurred_at	
synced_at	

USERS ||--o{ PRODUCTS (creates) · USERS ||--o{ TRANSACTIONS (performs) · PRODUCTS ||--o{ TRANSACTIONS (receives)



Oracle SQL Schema

```
CREATE TABLE users (  
  id          NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  email       VARCHAR2(160) UNIQUE NOT NULL,  
  password_hash VARCHAR2(255) NOT NULL,  
  full_name    VARCHAR2(120) NOT NULL,  
  role        VARCHAR2(20) DEFAULT 'STAFF' CHECK (role IN ('ADMIN','STAFF')),  
  is_active   NUMBER(1) DEFAULT 1 CHECK (is_active IN (0,1))  
);
```

```
CREATE TABLE products (  
  id          NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  sku         VARCHAR2(64) UNIQUE NOT NULL,  
  barcode     VARCHAR2(64),  
  name        VARCHAR2(200) NOT NULL,  
  cost_price  NUMBER(12,2) NOT NULL,  
  sell_price  NUMBER(12,2) NOT NULL,  
  low_stock_threshold NUMBER DEFAULT 5,  
  expiry_date DATE,  
  created_by  NUMBER REFERENCES users(id)  
);
```

```
CREATE TABLE transactions (  
  id          NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  client_txn_id VARCHAR2(40) UNIQUE NOT NULL,  -- idempotency key  
  product_id  NUMBER REFERENCES products(id),  
  user_id     NUMBER REFERENCES users(id),  
  txn_type    VARCHAR2(10) CHECK (txn_type IN ('IN','OUT')),  
  quantity    NUMBER(12,3) NOT NULL,  
  unit_price  NUMBER(10,2) NOT NULL,  
  occurred_at TIMESTAMP DEFAULT SYSTIMESTAMP  
);
```



Development Timeline





SWOT Analysis

STRENGTHS

- 100% offline uptime via IndexedDB + Service Workers
- Ultralight — zero JavaScript framework overhead
- Conflict-free transactions via client UUID idempotency

WEAKNESSES

- Database updates rely on client sync triggers
- Camera barcode detection needs modern browser

OPPORTUNITIES

- Multi-terminal retail POS expansion
- Local hardware receipt printer integration

THREATS

- Device cache-clear could affect unsynced data
- Evolving browser security for camera feeds



Project Cost Analysis (BDT)

DEVELOPMENT PHASE	MARKET ESTIMATE (BDT)	ACTUAL COST (BDT)
Design & UI/UX	10,000 – 15,000	0 (Student)
Frontend Development	45,000 – 60,000	0 (Student)
Backend Development	60,000 – 80,000	0 (Student)
Testing & QA	15,000 – 25,000	0 (Student)
Production Hosting	5,000 – 10,000	0 (Local machine)
Oracle Database	12,000 – 24,000	0 (XE 21c free)
Total	1,47,000 – 2,14,000	0 BDT

Thank You

Bazar-Trace is a fully functional, offline-resilient inventory application.

Built with pure vanilla code and relational Oracle schemas —
proving modern Web APIs can replace heavy commercial frameworks.

PRESENTER

Arif Hossen

ID: 231010687

SUPERVISOR

Reduanul Bari Subhon sir

MENTOR

Israt Jahan miss

I am now ready to take any questions from the panel.